GIBOU Adélie
ROSSIAUD Bastien

PROJET TSI

Table des matières

Introduction 2
Architecture du projet 2
Librairies utilisées 3
Structure du code et architecture logicielle 3
Gestion des collisions 4
Gestion de la physique 4
Gestion de la Caméra 5
Screenshots du jeu 5

Introduction

L'objectif de ce projet de synthèse d'image est de développer un jeu 3D en Python en utilisant OpenGL.
Le but du jeu est de collecter cinq fruits disposés aléatoirement dans un environnement 3D le plus rapidement possible.
Au lancement du programme principal, un écran d'introduction présente les différents fruits à récupérer. L'appui sur la touche Entrée démarre la partie et lance le chronomètre. Une fois la partie commencée, les fruits sont répartis aléatoirement sur différentes plateformes. Le joueur contrôle un personnage qui peut se déplacer dans l'espace 3D pour collecter les fruits grâce aux commandes suivantes :

- Haut ou Z : avancer
- Bas ou S : reculer
- Droite ou D : orienter la caméra vers la droite
- Gauche ou Q : orienter la caméra vers la gauche
- Espace : sauter
- R : réinitialiser la partie en cours
- Entrée : lancer une nouvelle partie
- Échap : quitter le programme

Une fois les cinq fruits récoltés, le joueur doit revenir sur la base afin de terminer sa partie et le chronomètre est alors arrêté et le temps affiché dans le terminal.

Architecture du projet

Le projet est développé en Python avec la bibliothèque OpenGL. Le code est organisé en plusieurs modules pour séparer la logique du jeu et le rendu graphique. Voici la structure des fichiers principaux :

- **main.py** : lance le jeu,
- **game.py** : contient la boucle de jeu principale et instancie tous les objets (voir Structure du code et architecture logicielle),
- **tools.py** : fonctions utilitaires pour charger les données de sommets et indices dans un VAO/VBO, ainsi que les textures,
- **Camera.py** : gère la caméra et son placement en fonction des objets présents,
- **collision.py** : détecte et résout les collisions entre 2 cubes,
- **binds.py** : gère les commandes clavier via les callbacks GLFW,
- **Dossier object** :
    - **cube.py** : génère un cube, représentant le personnage du jeu,
    - **mur.py** : génère un mur,
    - **sol.py** : génère un sol,
    - **sphere.py : génère** une sphère, représentant un fruit à collecter,

    - **3d_object.py** : classe générique pour gérer l'apparition des différents objets, leurs déplacements et l'animation des fruits,
- **Dossier shaders** :
    - **phong.vert** : vertex shader utilisant l'illumination de Phong,
    - **phong.frag** : fragment shader pour le shading des objets,
- **Dossier textures** : contient toutes les images appliquées aux objets pour améliorer l'aspect visuel.

Librairies utilisées

Le projet nécessite les librairies suivantes :

- **OpenGL** : pour les appels aux fonctions OpenGL,
- **glfw** : pour la gestion de la fenêtre, des événements et des entrées clavier,
- **numpy** : pour la manipulation efficace des tableaux,
- **pyrr** : pour la gestion des matrices et des transformations 3D,
- **PIL** : pour le chargement et la manipulation des images,
- **os** : pour la gestion des chemins de fichiers et la vérification de leur existence.

Structure du code et architecture logicielle

Chaque objet, outil ou méthode différent utilisé est séparé dans un fichier individuel réservé à cet effet. Ainsi, le projet est organisé et chaque élément est facilement accessible. Le programme game.py a le rôle de regrouper tous ces éléments et de les faire fonctionner ensemble. Dans ce programme, nous avons mis en œuvre une classe Game orientée objet dont le but est de générer le jeu final.

Chaque méthode va traiter une partie différente du projet, que cela soit côté graphique ou côté scientifique avec le calcul de différents paramètres.
Afin de pouvoir gérer tous les objets, la méthode *init_data* va initialiser les différents éléments. Chaque objet différent possédera un vao unique. Cela permet d'économiser de la mémoire et de la puissance de calcul tout en ayant la possibilité de créer une catégorie d'objets variés. Tous les vbo des objets sont importés des différents fichiers créer pour les types d'objet. Ils sont générés par des fonctions génériques qui laissent un degré de liberté quant aux différents paramètres de l'objet. Ces fonctions ont aussi le rôle de créer les maillages adéquats pour la mise en forme de l'objet.

Après que les vao, les vbo et les maillages des objets aient été créés, chaque objet est intégré comme étant un objet de la classe Object_3d. Cette technique permet de généraliser l'utilisation et la mise en œuvre d'un objet peu importe la forme ou les caractéristiques. Cela permet aussi au niveau de la programmation et de l'architecture de gérer un type d'élément objet_3d et de bien faire la différence avec d'autres éléments

comme la gravité ou les collisions. Chaque objet est donc défini selon des critères modifiables ce qui permet de créer beaucoup d'objets rapidement et efficacement.

La gestion des shaders vertex phong et fragment phong est réalisée dans la méthode *init*. Plusieurs méthodes permettent de gérer le placement aléatoire des fruits ou des murs lors du lancement de la partie.

Gestion des collisions

Pour gérer les collisions dans le jeu, nous utilisons la méthode AABB (Axis-Aligned Bounding Box). Chaque objet est entouré par une boîte alignée sur les axes X, Y et Z, ce qui simplifie la détection des collisions.
La fonction *check_collision_cc()* vérifie si ces boîtes se chevauchent sur les trois axes en comparant leurs bornes minimales et maximales. Une collision est détectée dès lors que les intervalles se recouvrent simultanément sur tous les axes.
La fonction *find_collision_cc()* calcule l'ampleur du chevauchement sur chaque axe et identifie celui où le recouvrement est le plus faible. Cet axe est ensuite utilisé pour résoudre la collision.
Enfin, *resolve_collision_cc()* ajuste la position de l'objet en collision sur cet axe et met sa vitesse correspondante à zéro, afin d'empêcher tout chevauchement.
Cette méthode assure une détection et une résolution de collisions simples et efficaces dans un environnement 3D.

Gestion de la physique

La physique appliquée à ce jeu consiste à appliquer la force de gravité sur les objets et plus principalement sur le cube qui nous sert d'avatar pour jouer au jeu.

La gravité étant une force, le but va être d'appliquer une force constante vers le bas afin que les objets tombent ou rencontrent d'autres objets avec qui il y aurait des collisions permettant d'arrêter la chute des objets (comme un sol par exemple). Pour appliquer cette force, nous utilisons des équations de physique et plus particulièrement le principe fondamental de la dynamique. Ce principe énonce que la somme des forces appliquées sur un objet est égale au produit de la masse et de l'accélération de l'objet tel que :

$$\vec{F} = m \cdot \vec{a}$$

Pour simuler la gravité, on applique une force constante avec un vecteur gravité jouant le rôle du vecteur accélération tel que $\vec{g} = -9{,}81\ m/s^2$.

En plus du principe fondamental de la dynamique, nous utilisons le schéma d'Euler explicite qui est une méthode qui permet de résoudre des équations différentielles. On approxime donc :

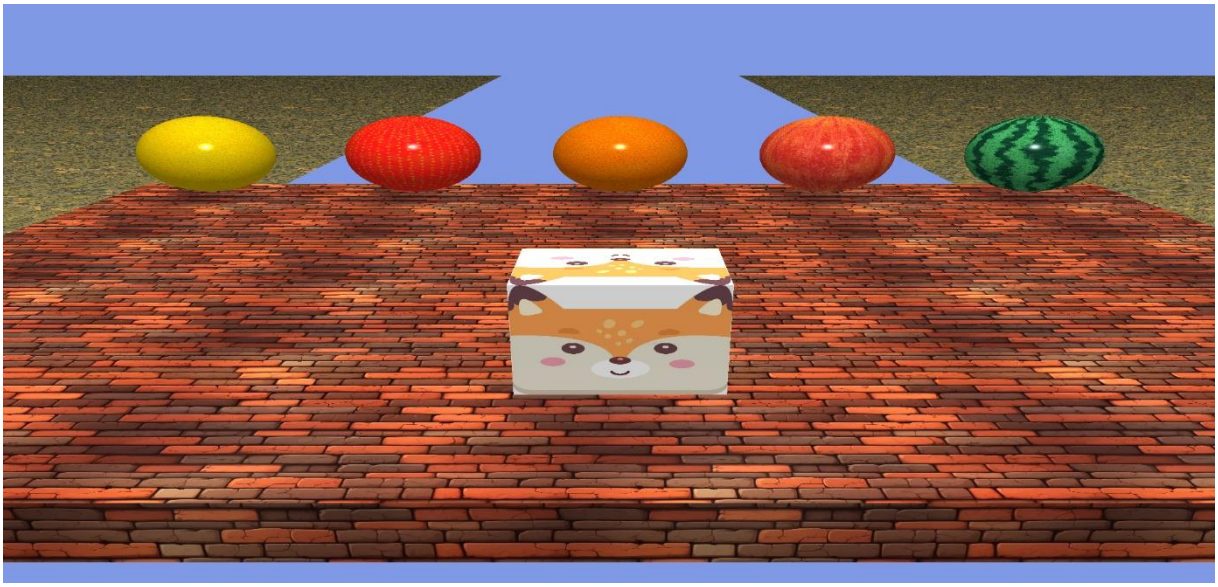
$$\vec{v}(t + \Delta t) = \vec{v}(t) + \vec{a}(t)\Delta t$$
$$\vec{p}(t + \Delta t) = \vec{p}(t) + \vec{v}(t)\Delta t$$

C'est avec ces deux méthodes que l'on peut simuler la gravité et des grandeurs physiques dans notre projet.

Gestion de la Caméra

Un des problèmes que nous avons rencontré avec la caméra est que quand notre avatar était derrière un mur, la caméra était soit derrière le mur soit buggée dans le mur ce qui résultait à un manque de vision sur l'avatar et donc une gestion des commandes compliquée. Afin de pallier ce problème, nous avons créé une classe caméra qui actualise la position de la caméra en fonction des objets entre l'avatar et la caméra. Pour se faire nous avons implémentés trois fonctions qui permettent de calculer la position, de l'actualiser et de vérifier les collisions entre la caméra et les différents objets. Afin de vérifier les collisions, nous créons une classe temporaire qui permet de faire passer la caméra comme un objet de la classe 3d.

Screenshots du jeu



Nous pouvons observer qu'au lancement du jeu, l'avatar que nous incarnons (ici le cube à tête de renard) se trouve sur une plateforme avec cinq fruits devant lui. Ces cinq fruits

sont les fruits à récupérer au lancement du jeu. Ils seront placés aléatoirement quand le jeu sera lancé.



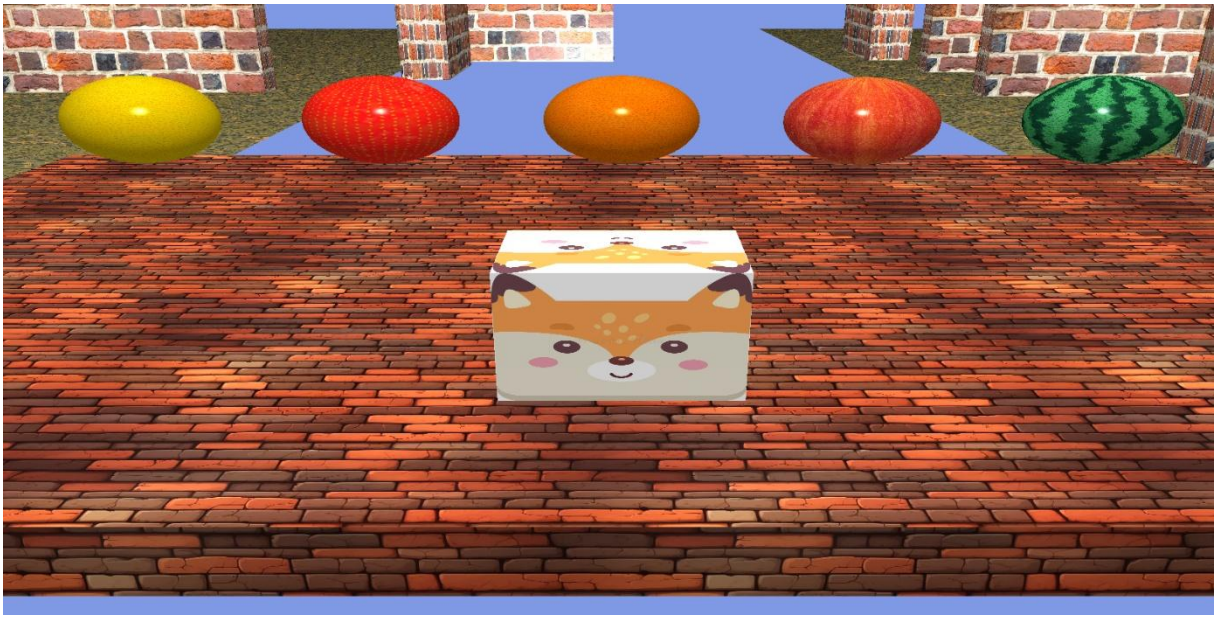
Une fois le jeu lancé, les fruits sont cachés et le but est d'aller les chercher. Cependant, des murs sont aussi générés afin d'obstruer le passage et de rendre la tâche plus difficile.



On peut voir que les fruits sont cachés derrière les murs.



Le point de vue de la caméra est modifié à proximité des murs.



Quand les cinq fruits sont récupérés, il faut retourner sur la plateforme centrale et marcher sur la case de fin pour de terminer le jeu.

```
Il ne reste plus que 4 fruits à ramasser !
Il ne reste plus que 3 fruits à ramasser !
Il ne reste plus que 2 fruits à ramasser !
Il ne reste plus que 1 fruits à ramasser !
Tous les fruits ont été ramassés il faut maintenant revenir à la base !
Tous les fruits ont été ramassés en 165.48 secondes !
```

Voici le terminal au cours et à la fin de la partie.